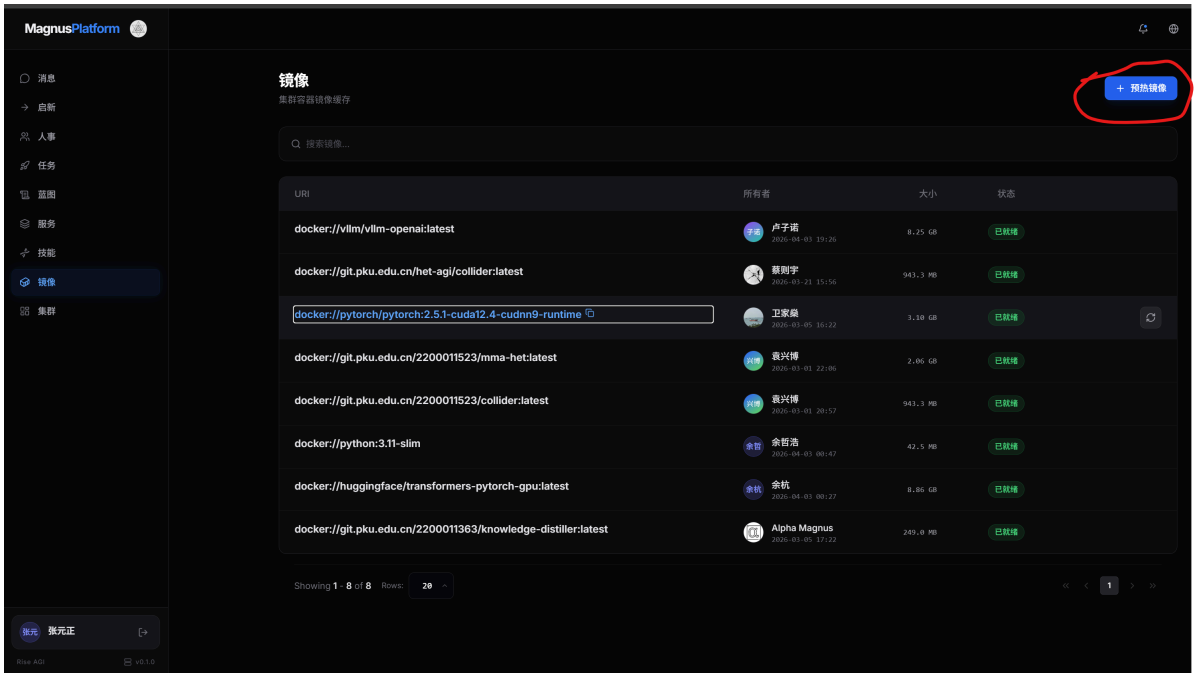
magnus输入输出方法

一 预热



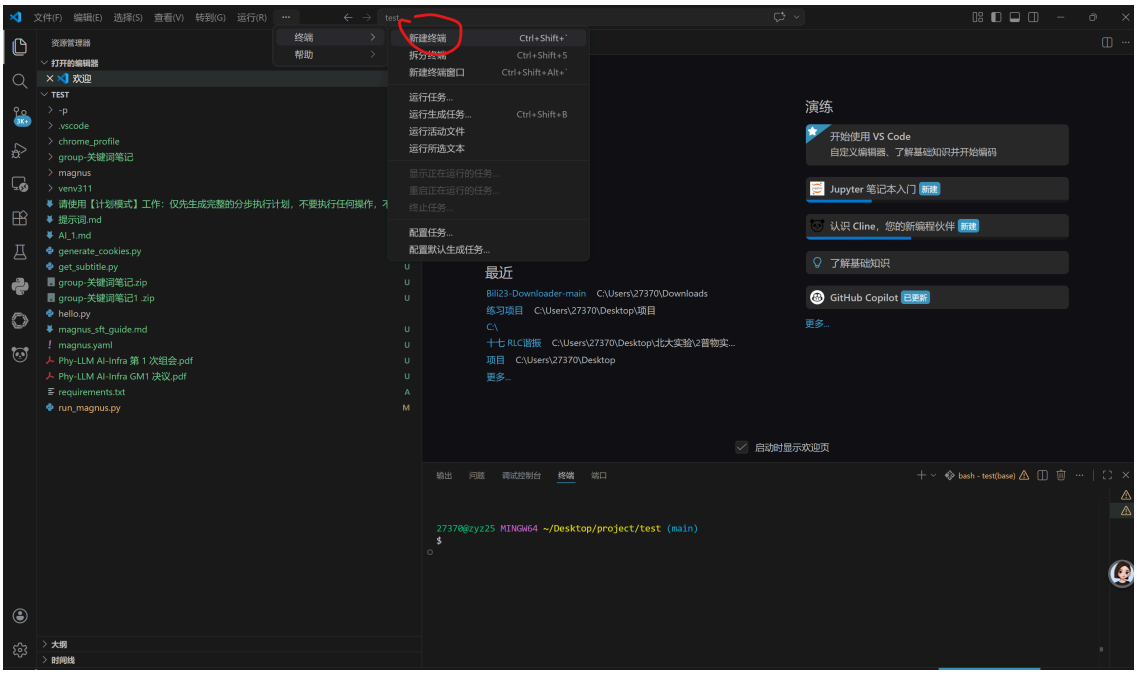
选择需要的环境(图中为pytorch); 非管理员无法添加环境(GPT-2够用了)

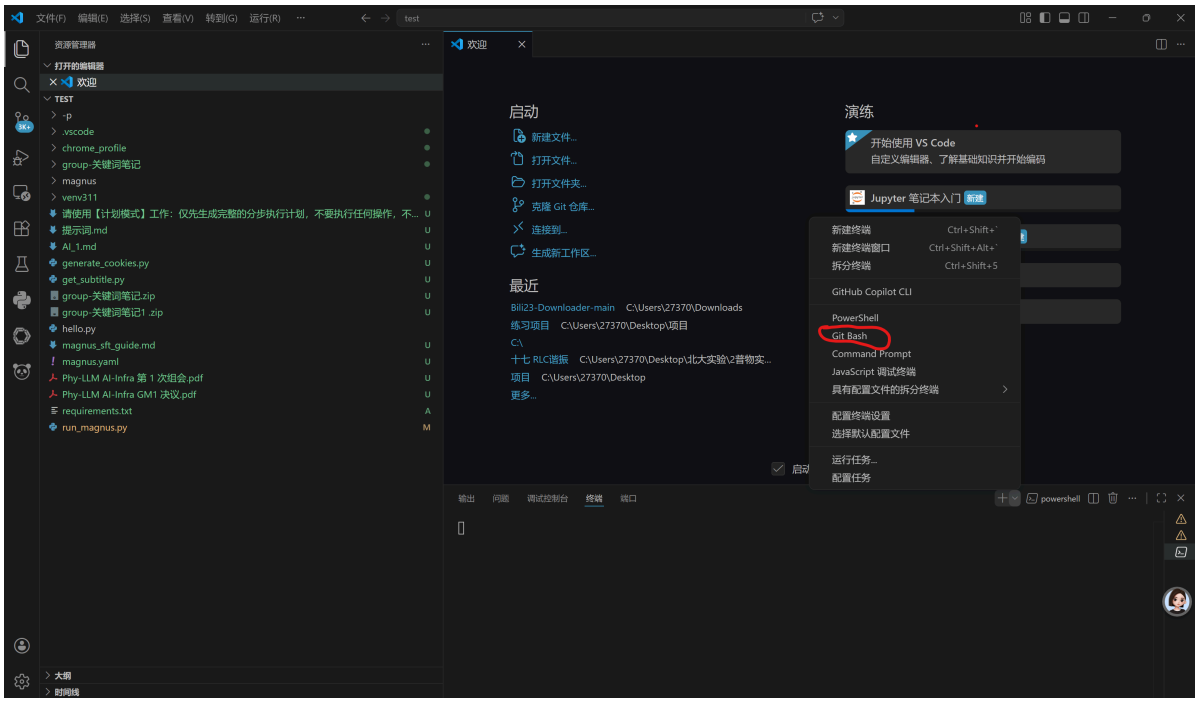
二 提交文件到github

二 提交文件到 GitHub 完整操作笔记(所有均使用GIT BASH)

前提准备

1. 本地已安装 Git 工具
2. 已配置 GitHub 账号（用户名、邮箱），或已配置 SSH 密钥免密登录
3. 拥有 GitHub 远程仓库地址（HTTPS / SSH 格式）






```
# ==============================================
# Git 从配置到提交GitHub 全套命令（带注释）
# ==============================================

# 1. 全局配置Git身份（首次使用必做，关联GitHub账号）
git config --global user.name "你的GitHub用户名"    # 设置用户名（和GitHub一致）
git config --global user.email "你的GitHub绑定邮箱" # 设置邮箱（和GitHub一致）
git config --list                                   # 查看配置是否生效

```

```
# 2. 克隆远程仓库到本地（仅第一次操作需要）
git clone 你的GitHub仓库地址   # 复制GitHub仓库的HTTPS/SSH地址粘贴此处
cd 仓库文件夹名                # 进入克隆好的本地仓库目录

# 3. 拉取远程最新代码（每次修改前必做，防止代码冲突）
git pull

# --------------------------
# 【手动操作】在此处修改、新增、删除本地文件
# --------------------------

# 4. 将所有修改的文件添加到暂存区
git add .                      # . 代表添加当前目录所有修改文件

# 5. 提交到本地Git仓库（必须写备注，描述本次修改内容）
git commit -m "本次修改的备注信息"  # 示例：git commit -m "更新笔记内容"

# 6. 推送到GitHub远程仓库（完成上传）
git push

# 辅助命令：随时查看文件状态/提交状态
git status
```

完整操作步骤

2. 将远程仓库克隆到本地

首次操作时，把 GitHub 上的仓库下载到本地电脑：

```
git clone 远程仓库地址
```

示例：

```
git clone https://github.com/xxx/xxx-repo.git
```

执行后会在当前目录生成项目文件夹，进入该文件夹：

```
cd 仓库文件夹名
```

3. 拉取远程仓库最新代

在修改文件前，先同步远程最新内容，避免版本冲突：

```
git pull
```

5. 修改本地仓库文

- 新增、删除、编辑项目内的代码/文档等文件
- 确认修改完成后，准备提交

4. 将修改文件加入暂存区

方式1：添加指定文件

```
git add 文件名
```

方式2：添加所有修改文件（常用）

```
git add .
```

6. 提交暂存区内容到本地仓库

必须添加提交说明，清晰描述本次修改内容

```
git commit -m "本次提交的说明信息"    \回车
git push
```

示例：

```
git commit -m "fix: 修正README文档格式"   \回车
git push
```

注意bash只能逐行输入, 所以git commit -m \回车　后会有红色报错, 不用管

执行完成后，刷新 GitHub 仓库页面，即可看到更新的文件。

补充常用说明

- 若首次推送，部分情况需指定分支：`git push origin main`（主分支名可能为 `main` / `master`）
- 提交前可使用 `git status` 查看文件修改状态(未`add`文件标红)，`git log` 查看提交记录
- 删改文件可能出现弹窗登录网站二次验证, 跟着要求来就行了

三 运行

克隆任务

任务信息

任务名称 *

magnus_fisrt_test

描述（可选）

一个示例代码来熟悉Magnus使用方法

代码来源

命名空间

Rise-AGI

仓库名称

PHY-LLM-Basic-ALgorithm

扫描仓库

分支

main

提交

最新提交 (HEAD)

任务调度

任务优先级

B2 - 次优可抢

计算资源

CPU 核心数

4

内存

1600M

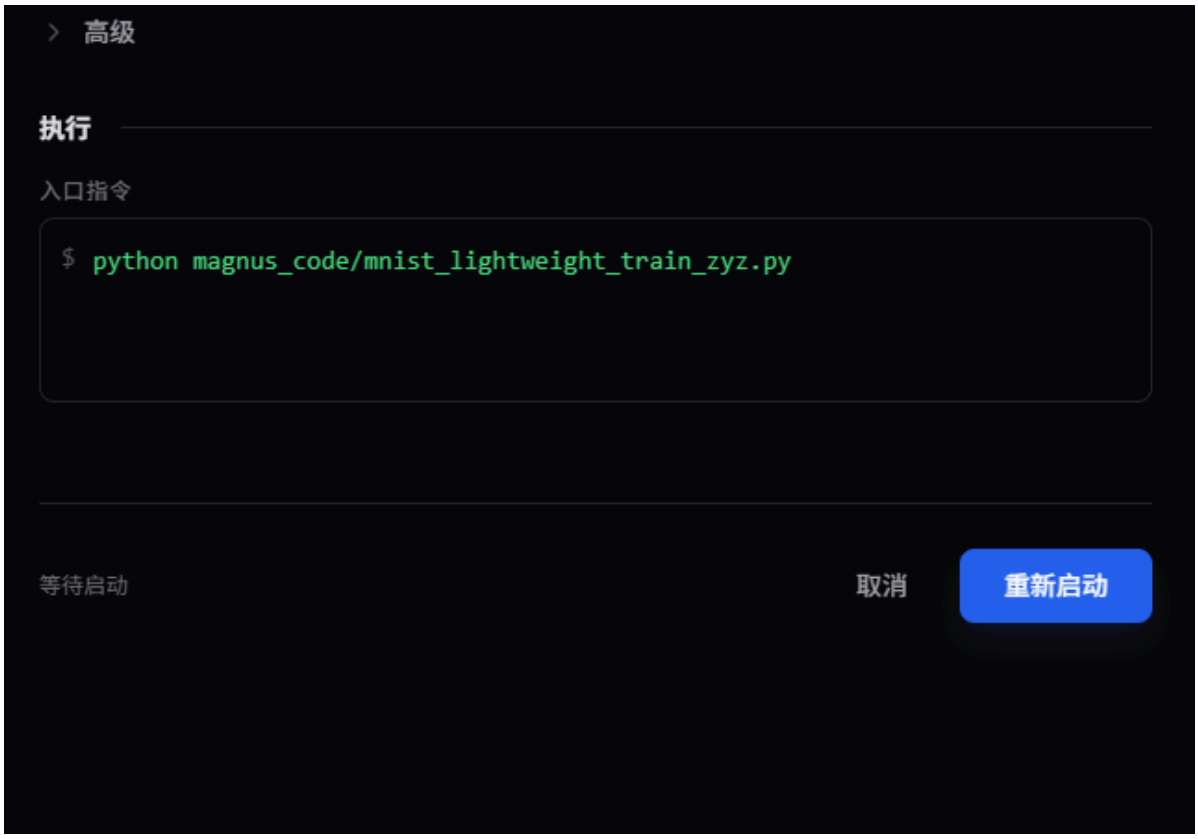
设为 0 使用默认值 (4)。

GPU 加速器

NVIDIA A100 80GB PCIe

GPU 数量

2



- 需要填仓库名(在github中查看), 选择`最新提交`
- 资源和优先级按需求填(记得去magnus首页集群看剩多少), A级需要正当理由(写在名称和描述中)
- 入口命令相当于cmd, 但几乎没有高级功能, 环境是整个仓库https://github.com/Rise-AGI/PHY-LLM-Basic-Algorithmgithub.com/Rise-AGI/PHY-LLM-Basic-Algorithm
- 入口命令直接 **python + 文件在仓库中的相对地址** , 比如图中文件的绝对地址为

  https://github.com/Rise-AGI/PHY-LLM-Basic-Algorithmgithub.com/Rise-AGI/PHY-LLM-Basic-Algorithm/magnus_code/mnist_lightweight_train_zyz.py只需填写/magnus_code/mnist_lightweight_train_zyz.py
- magnus只提供print式输出, 过程中创建和修改的文件不会主动输出, 需要在python运行过程中用python提交至其他网站下载(已经测试过, 入口指令不支持配置库, http连接, SSH连接 ;这些都只能在python文件内部执行)

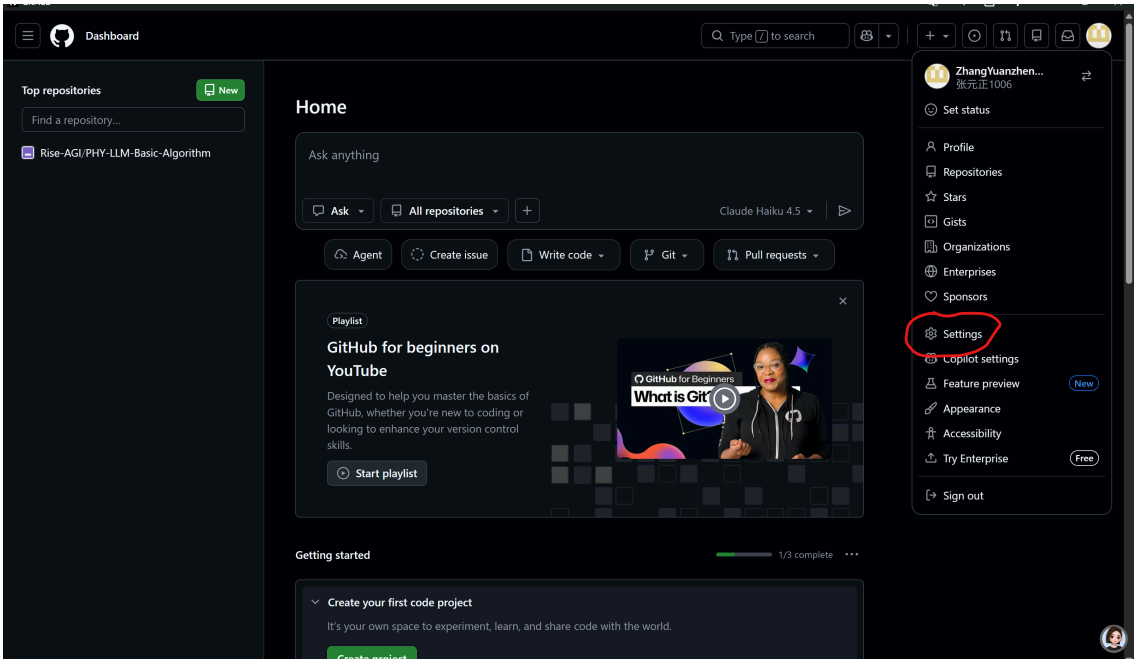
四 文件输出(直接上传到github仓库)

上传文件到github需要口令(详细见下), 但是github不允许在上传的python代码中出现口令(因为这是公开的仓库, 获得口令相当于获得了你的账号密码, 实际上传会被github拒绝), 所以 我们使用入口命令环境变量传入口令到文件

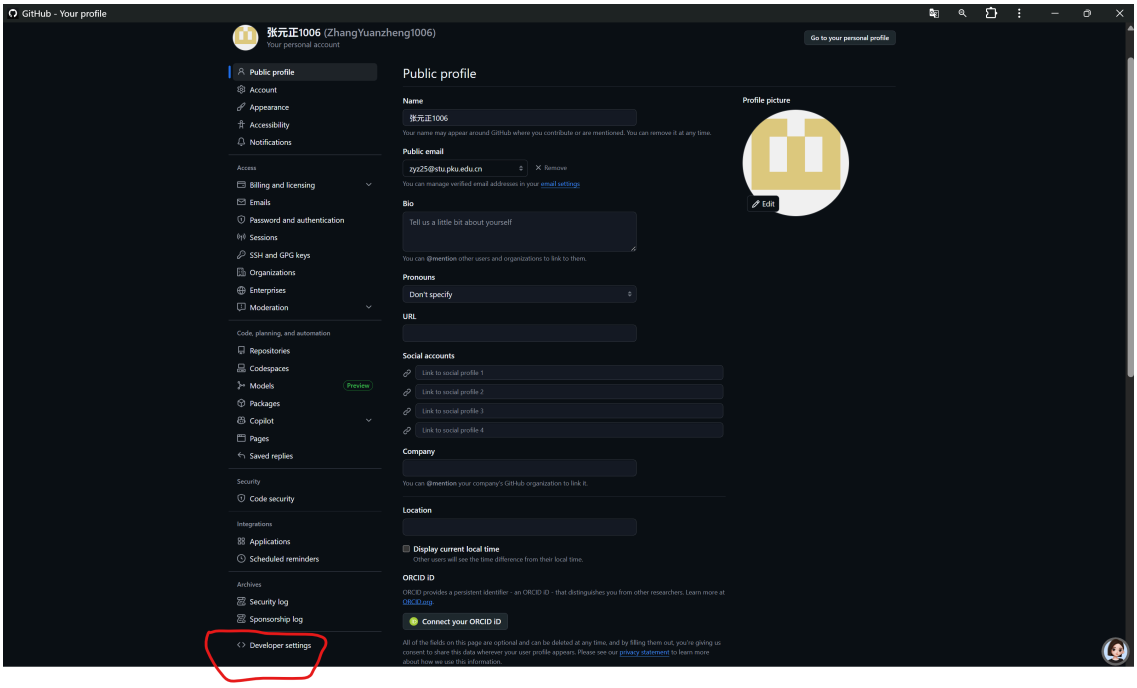
1 GitHub 令牌创建

必须创建**经典令牌（Tokens (classic)）**，精细令牌权限不足会导致403，步骤如下：

1. 打开GitHub → 右上角头像 → `Settings`

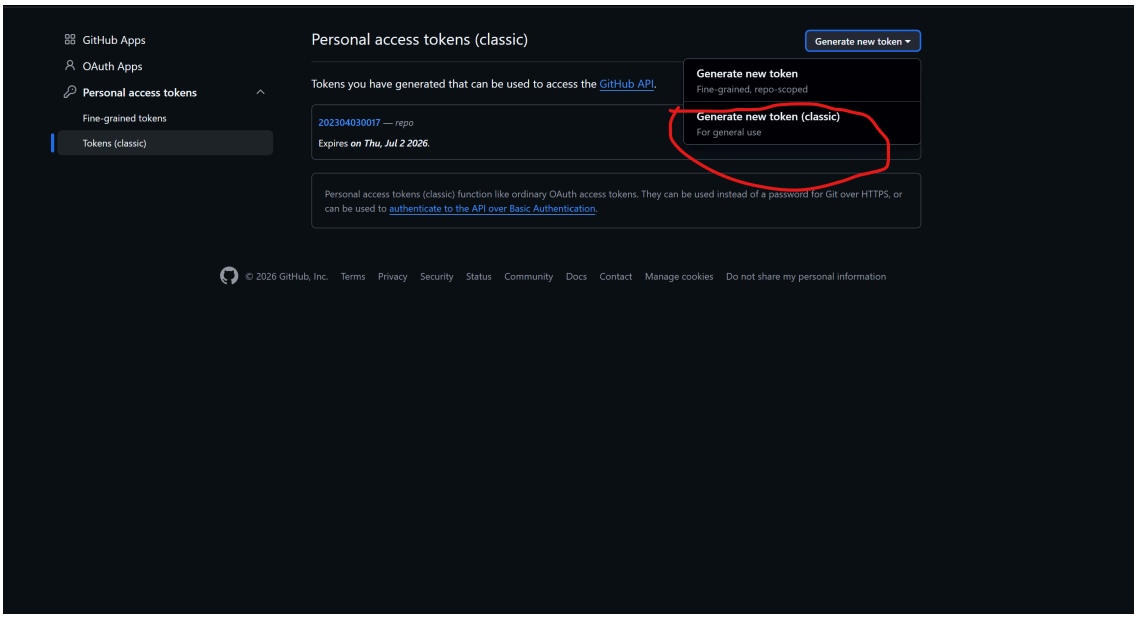


→ 左侧拉到底 `Developer settings`

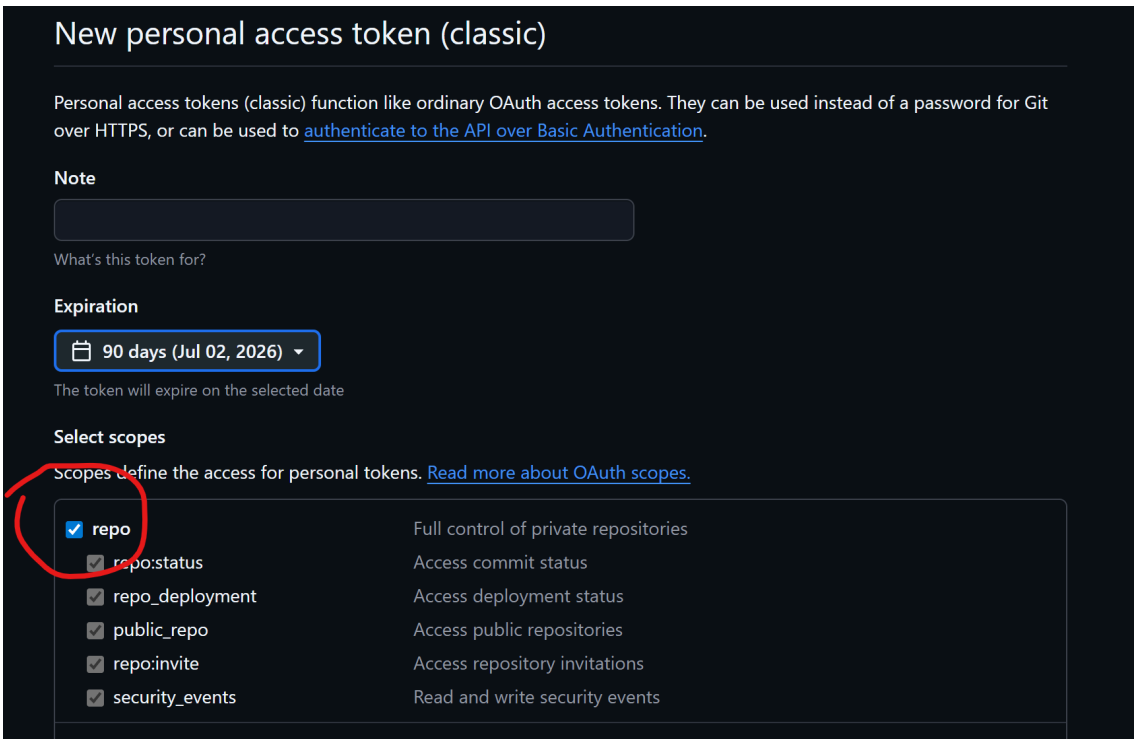


→ `Personal access tokens` → `Tokens (classic)`

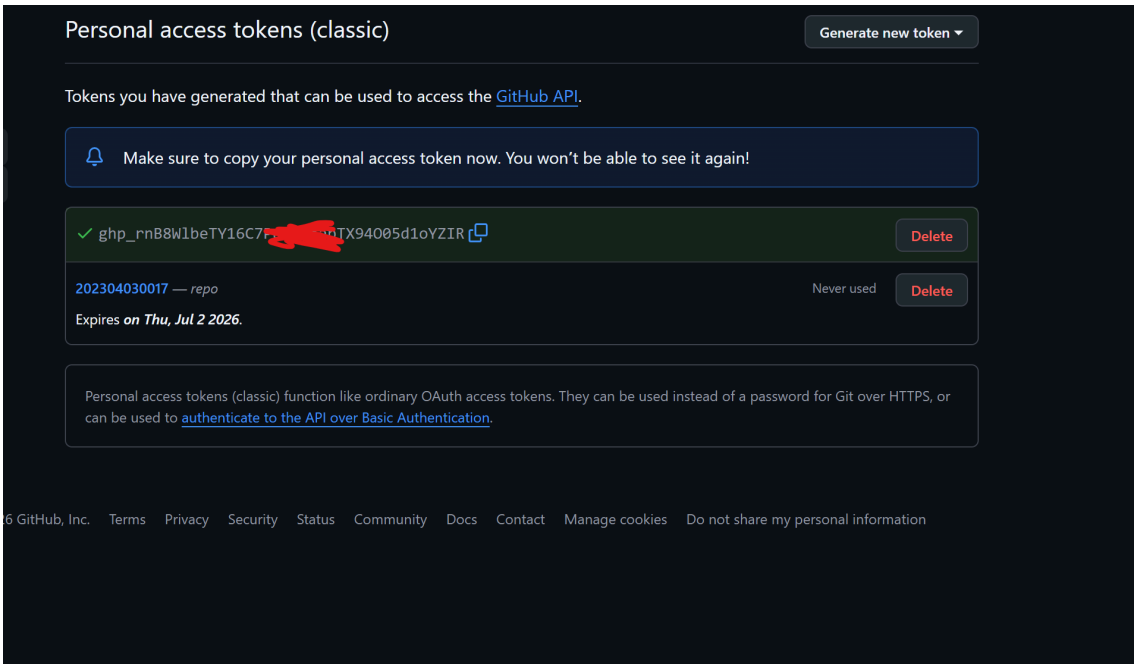
2. 点击 `Generate new token (classic)`，填写备注（如`Magnus-upload-model`）



3. **权限勾选**：全选`repo`（仓库相关所有权限），有效期按需选择



4. 生成后**立即复制并另存令牌!!!**（格式：`github_pat_xxxx`/`ghp_xxxx`），**仅显示一次!!!**，妥善保存,不要泄露(图中打码部分ghp_XXXX)



2 标准上传代码

(详细示例版见[https://github.com/Rise-AGI/PHY-LLM-Basic-Algorithmgithub.com/Rise-AGI/PHY-LLM-Basic-Algorithm/magnus_code/mnist_lightweight_train_zyz.py](https://github.com/Rise-AGI/PHY-LLM-Basic-Algorithm/magnus_code/mnist_lightweight_train_zyz.py))

```python
# ==================== GitHub 通用上传函数（核心框架）====================
import base64
import json
import urllib.request
import os
from urllib.error import HTTPError

def push_git(local_file_path: str, github_file_path: str):
    """
    【Magnus 专用】将本地文件一键上传到 GitHub 仓库
    :param local_file_path:  Magnus 容器内的文件路径（如 ./model.pth）
    :param github_file_path:  GitHub 仓库内的目标路径（如 magnus_code/model.pth）
    """
```

```
    # ==================== 固定配置（无需修改）====================
    GITHUB_TOKEN = os.getenv("GITHUB_TOKEN", "").strip()
    REPO = "Rise-AGI/PHY-LLM-Basic-Algorithm"
    BRANCH = "main"
    COMMIT_MSG = "auto upload file from Magnus"

    # 校验 Token
    if not GITHUB_TOKEN:
        print("⚠️ 未检测到 GITHUB_TOKEN 环境变量，跳过上传")
        return
    # 校验本地文件
    if not os.path.exists(local_file_path):
        print(f"❌ 本地文件不存在：{local_file_path}")
        return

    # 获取已存在文件 SHA（用于覆盖上传）
    def _get_file_sha():
        url = f"https://api.github.com/repos/{REPO}/contents/{github_file_path}"
        try:
            req = urllib.request.Request(url)
            req.add_header("Authorization", f"token {GITHUB_TOKEN}")
            with urllib.request.urlopen(req) as f:
                return json.loads(f.read()).get("sha")
        except:
            return None

    # 执行上传
    try:
        # 文件编码
        with open(local_file_path, "rb") as f:
            content = base64.b64encode(f.read()).decode("utf-8")

        # 请求参数
```

```
        url = f"https://api.github.com/repos/{REPO}/contents/{github_file_path}"
        data = {"message": COMMIT_MSG, "content": content, "branch": BRANCH}

        # 覆盖文件处理
        sha = _get_file_sha()
        if sha:
            data["sha"] = sha
            print("ℹ️ 检测到同名文件，执行覆盖上传")

        # 发送请求
        req = urllib.request.Request(url, data=json.dumps(data).encode(), method="PUT")
        req.add_header("Authorization", f"token {GITHUB_TOKEN}")
        req.add_header("Content-Type", "application/json")
        req.add_header("User-Agent", "Magnus-Upload")

        with urllib.request.urlopen(req):
            print("✅ 上传成功！")
            print(f"本地：{local_file_path}")
            print(f"远程：{github_file_path}")

    except HTTPError as e:
        print(f"❌ 上传失败（HTTP {e.code}）：权限/令牌/路径错误")
    except Exception as e:
        print(f"❌ 上传异常：{str(e)}")

# ==================== 使用示例（替换路径即可）====================
if __name__ == "__main__":
    # 示例1：上传模型文件
    push_git(
        local_file_path="./mnist_light_model.pth",  # Magnus 内文件路径
```

```
    github_file_path="magnus_code/mnist_light_model.pth"  # GitHub 目标路径
    )

    # 示例2：上传日志/文本等任意文件
    # push_git("./log.txt", "magnus_code/logs/run.log")
```

3 入口指令

将`<你的GitHub经典令牌>`替换为1.1中生成的令牌，***必须整行输入***，换行会导致Token失效：

```
GITHUB_TOKEN="<你的GitHub经典令牌>" python magnus_code/mnist_lightweight_train_zyz.py
```

五 完整运行流程（按步骤操作，无遗漏）

步骤1：本地提交代码到GitHub仓库

将2.1的代码保存到本地仓库`magnus_code/`目录下，执行以下命令推送到GitHub：

```

# 进入本地仓库目录
cd ~/Desktop/项目/PHY-LLM-Basic-Algorithm
# 添加代码文件
git add magnus_code/
# 提交备注
git commit -m "feat: 新增MNIST训练+自动上传GitHub代码"
# 推送到main分支
git push origin main
```

步骤2：Magnus平台启动任务

按2.3填写完所有配置后，点击**启动任务**，任务会进入`Pending(排队中)`，等待集群资源空闲后自动运行。

步骤3：查看运行日志，确认训练+上传成功

任务启动后，点击任务名称进入**详情页→控制台输出**，查看日志，成功标志：

1. 数据集下载：MNIST官方源404后自动切换备用源，最终显示`数据集加载完成：训练集60000张，测试集10000张`
2. 训练完成：显示`最终测试集准确率：98%+`、`训练完成！模型已保存至：./mnist_light_model.pth`
3. 上传成功：显示`✅ 模型上传GitHub成功！`

六 过程中典型错误&原因&解决方案（全踩坑整理，快速排障）

错误现象	核心原因	解决方案
`No such file or directory: xxx.py`	文件名/路径写错（Linux大小写敏感）；代码不在根目录却未加路径前缀	严格匹配GitHub仓库文件名（含后缀/大小写）；代码在`magnus_code/`则命令加前缀`magnus_code/`
`git/ssh/ssh-keyscan/curl: command not found`	Magnus容器为极简版，无任何系统工具，且无root权限无法通过apt安装	放弃系统命令，使用**纯Python实现**（本笔记代码已适配）
GitHub推送被拦截（GH013，检测到明文Token）	代码中直接写死GitHub令牌，触发GitHub安全扫描保护	代码中通过`os.getenv`从环境变量读取Token，本地/平台均不写明文Token
`NameError: name 'test_epoch' is not defined`	笔误，调用测试函数时写错函数名（实际函数名为test_model）	修正函数名，调用`test_model`（本笔记代码已修复）
`⚠️ 未检测到GitHub Token，跳过上传`	入口命令将Token和Python命令**换行输入**，环境变量失效	将Token和Python命令写在**同一行**（本笔记2.2命令格式）

错误现象	核心原因	解决方案
❌ 上传失败：`HTTP Error 403: Forbidden`	1. 令牌为精细令牌，repo权限不足；2. GitHub已存在同名文件，无sha值无法覆盖	1. 重新创建**经典令牌**并勾选全repo权限；2. 代码中添加sha值获取逻辑（本笔记代码已修复）
`dpkg: error: requested operation requires superuser privilege`	尝试apt安装git/ssh，Magnus容器无root权限，禁止安装软件	放弃apt安装，使用平台原生支持的Python内置库实现所有功能
MNIST下载显示`HTTP Error 404: Not Found`	MNIST官方源地址失效，代码会自动切换备用源	无需处理，属于正常现象，等待备用源下载完成即可
`WARNING: Overriding HOME environment variable with APPTAINERENV_HOME is not permitted`	Magnus容器系统默认提示，无实际影响	直接忽略，不影响训练/上传

七 关键注意事项（核心原则，必须遵守）

5.1 Magnus平台专属注意事项

1. 容器无**root/sudo权限**，**禁止任何apt/yum安装操作**，所有功能仅能通过Python内置库实现；
2. 入口命令**必须单行输入**，环境变量（如GITHUB_TOKEN）和运行命令不可换行，否则变量失效；
3. 内存严格控制在**申请内存以内**，避免OOM（本笔记代码超参数已做内存优化，峰值<600M）；
4. 任务配置的**容器镜像**必须和**预热的镜像**完全一致，否则会重新拉取导致任务启动失败；
5. 平台无文件下载功能，所有输出文件需通过代码自动上传到GitHub，不可依赖平台文件管理。

5.2 代码编写注意事项

1. 代码中**绝对禁止写清明文GitHub令牌/密钥**，避免触发GitHub安全扫描，导致本地推送被拦截；
2. 函数名/变量名需严格一致，避免笔误导致的NameError（本笔记代码已修复所有笔误）；
3. 数据加载器`num_workers`设置为2，避免过多进程占用内存，适配平台内存限制。

5.3 GitHub操作注意事项

1. 上传模型的令牌**必须为经典令牌（Tokens (classic)）**，精细令牌权限不足会导致403，无法上传；
2. 令牌必须勾选**全repo权限**，仅勾选部分权限会导致仓库写入失败；
3. 本地推送代码前，确保代码中无明文敏感信息（如令牌、密码），避免被GitHub拦截。

5.4 令牌安全注意事项

1. GitHub令牌相当于账号密码，**严禁分享给他人、严禁提交到代码仓库**；
2. 令牌有效期按需设置，短期使用可设置7天，避免令牌泄露导致安全风险；
3. 若令牌泄露，立即在GitHub上**撤销该令牌**，重新生成新令牌。

六、最终验证（确认任务完成，成果可复用）

1. **训练验证**：Magnus控制台日志显示训练完成，测试集准确率≥98%；
2. **上传验证**：打开GitHub仓库`Rise-AGI/PHY-LLM-Basic-Algorithm`，进入`magnus_code/`目录，能看到`mnist_light_model.pth`文件，即为上传成功；
3. **模型复用**：可将该模型文件下载到本地，通过`torch.load`加载，直接用于MNIST手写数字识别推理。

> （注：文档部分内容可能由 AI 生成）